

Training Math: Gradient Descent, Loss & Optimizers

The math shared by every model-training loop: gradient descent, loss functions, momentum, Adam, and regularization

Concept Guide 5 of 9 | Read before: [PyTorch](#) or [TF/Keras lab](#) | Companion to the [Zero-to-Hero](#) series

1. Gradient Descent: The Update Rule

Every model in this course — regression, neural nets, LLMs — is trained by the same core idea: nudge each parameter a small step opposite to its gradient, repeatedly, until the loss stops improving.

FORMULA — The update rule

$$\text{param} = \text{param} - \text{learning_rate} * \text{gradient}$$

learning_rate controls how big each step is

WORKED EXAMPLE — A single step by hand

param = 5.0, gradient = 2.0, learning_rate = 0.1

new_param = 5.0 - 0.1 * 2.0 = 5.0 - 0.2 = 4.8

Repeat this every step, with the gradient recomputed each time.

COMMON MISUNDERSTANDINGS

- Learning rate too LARGE: the parameter overshoots the minimum and can diverge (loss increases instead of decreases).
- Learning rate too SMALL: training converges correctly but painfully slowly.
- There is no universal 'correct' learning rate — it's tuned per problem, often starting around 0.001-0.01 for Adam.

PRACTICE THIS → [PyTorch_Zero_to_Hero.ipynb Ch.4-5](#); [TensorFlow_Keras_Zero_to_Hero.ipynb Ch.4](#)

2. Loss Functions: MSE and Cross-Entropy

The loss function is what gradient descent minimizes — it defines 'how wrong' the model currently is, in a single number.

FORMULA — Mean Squared Error (regression)

$$\text{MSE} = (1/n) * \text{sum}((y_{\text{true}} - y_{\text{pred}})^2)$$

squares the error so bigger mistakes are penalized disproportionately more

FORMULA — Cross-Entropy (classification)

$$\text{CE} = -(1/n) * \text{sum}(y_{\text{true}} * \log(y_{\text{pred}}))$$

for multi-class; y_{pred} comes from softmax; see the Math Foundations guide for the single-class version

MENTAL MODEL

MSE cares about DISTANCE (how far off is the number?). Cross-entropy cares about CONFIDENCE (how far off is the probability?). Using the wrong loss for the task — e.g. MSE for a classifier — trains the model toward the wrong objective.

PRACTICE THIS → [PyTorch_Zero_to_Hero.ipynb Ch.7-8](#); [TensorFlow_Keras_Zero_to_Hero.ipynb Ch.6-7](#)

3. Momentum: Remembering Past Gradients

Plain gradient descent (SGD) can zig-zag inefficiently across a loss surface. Momentum smooths this out by accumulating a running average of past gradients.

FORMULA — Momentum update

$$\text{velocity} = \text{momentum} * \text{velocity} + \text{gradient}$$

$$\text{param} = \text{param} - \text{learning_rate} * \text{velocity}$$

momentum is typically ~0.9; it 'remembers' the direction of recent steps

MENTAL MODEL

Think of a ball rolling downhill: momentum lets it keep moving through small bumps and flat regions instead of getting stuck, and it smooths out oscillation across steep, narrow valleys in the loss surface.

PRACTICE THIS → [PyTorch_Zero_to_Hero.ipynb Ch.5](#)

4. Adam: Adaptive Learning Rates

Adam (the default optimizer in almost every lab in this course) combines momentum with a PER-PARAMETER adaptive learning rate, based on how large that parameter's recent gradients have been.

FORMULA — Adam's core idea (simplified)

m = running average of gradients (like momentum)
v = running average of squared gradients (tracks gradient
MAGNITUDE)
param = **param** - **learning_rate** * **m** / (**sqrt(v)** + **epsilon**)
dividing by **sqrt(v)** shrinks the effective step for parameters with consistently large gradients

MENTAL MODEL

Adam automatically gives smaller steps to parameters that have been changing wildly (to stabilize them) and larger steps to parameters that have been changing slowly (to speed them up) — this adaptivity is why Adam usually converges faster than plain SGD with little manual tuning.

PRACTICE THIS → [PyTorch_Zero_to_Hero.ipynb Ch.5, Ch.7](#)

5. Regularization: Fighting Overfitting

A model that fits training data too precisely (including its noise) will generalize poorly. Regularization techniques deliberately constrain the model to prevent this.

FORMULA — L2 weight penalty (weight decay)

Loss_total = **Loss_data** + **lambda** * **sum(w^2)**
lambda controls how strongly large weights are penalized

DIAGRAM — Dropout (used in deep networks)

training: randomly ZERO OUT a fraction of neurons each forward pass
inference: use ALL neurons (with an adjustment for the scale difference)
effect: the network can't rely on any single neuron, forcing redundant, robust features

COMMON MISUNDERSTANDINGS

- Regularization is a TRAINING-time technique — dropout must be disabled during evaluation/inference (this is exactly why `model.eval()` / `model.train()` exist in PyTorch).
- Too much regularization causes UNDERfitting — the model becomes too constrained to learn the real pattern at all.

PRACTICE THIS → [PyTorch_Zero_to_Hero.ipynb Ch.7](#); [Classical_ML_Zero_to_Hero.ipynb Ch.4](#)

6. Normalization: Standardizing Inputs and Activations

Unscaled inputs (or wildly varying internal activations) make training unstable and slow. Normalization keeps values in a consistent, well-behaved range.

FORMULA — Feature standardization (recap)

$$\mathbf{x_normalized} = (\mathbf{x} - \text{mean}) / \text{std_dev}$$

applied to INPUTS before training; see the Math & Statistics guide

FORMULA — Layer normalization (inside a network)

$$\mathbf{y} = (\mathbf{x} - \text{mean}(\mathbf{x})) / \text{sqrt}(\text{var}(\mathbf{x}) + \text{epsilon})$$

normalizes each example's own activations, used heavily inside Transformer blocks

MENTAL MODEL

Without normalization, one large-scale feature can dominate the loss landscape, making gradient descent take a very indirect, slow path to the minimum. Normalization reshapes the loss surface into something closer to a smooth bowl, which gradient descent navigates far more efficiently.

PRACTICE THIS → [Transformers_Attention_Zero_to_Hero.ipynb Ch.7 \(LayerNorm\)](#);
[PyTorch_Zero_to_Hero.ipynb Ch.7](#)